ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería de Sistemas · Carrera de Ciencia de Datos e Inteligencia Artificial

INFORME DE PRÁCTICA Nº 1

Consolidación de reportes de ventas

Tópico _____del PEA · Asignatura IDSD422 — Estadística y Programación para Ciencia de Datos I

Estudiante(s)	López Patiño Ethan Jeriel — usuario de GitHub: Jeriel2
Docente	Cristian Suarez
Períodoacadémico	2026-B
Fechadeentrega	19/10/2026
Repositorio (fork)	github.com/Jeriel2/IDSD422-lab1-lectura-datos

1 · Introducción

En la vida profesional no siempre trabajaremos con los mismos tipos de archivos, y justo el caso de la empresa RetailAndina S.A., nos lo demuestra. En este caso el reporte semanal en vez de venir en un solo formato les llego en tres distintos, esto debido a que provienen de tres sistemas independientes:

Tabla 1

Sistemas de origen de cada formato y su contenido

Sistema	Formato	Contenido
ERP de sucursales	ventas.csv	Exportación tabular del sistema de facturación
Plataforma de fidelización	ventas.json	Respuesta del servicio web (API) con la misma información
Finanzas	ventas.xlsx	Consolidado que la administración prepara en Excel

Por ello la gerencia ha solicitado al equipo un primer análisis: confirmar que los tres reportes describen la misma información, describir su estructura (filas, columnas y tipos de datos) y calcular indicadores iniciales para la reunión del lunes.

2 · Objetivos

- **Objetivo general:** Consolidar el reporte semanal de ventas de RetailAndina S.A., recibido en tres formatos (CSV, JSON y Excel), verificando con Python y R que los tres describen la misma información y calculando los indicadores iniciales que pidió la gerencia.
- **Objetivos específicos:**

1. Leer ventas.csv, ventas.json y ventas.xlsx tanto en Python (pandas) como en R (readr, jsonlite y readxl).
2. Verificar las dimensiones (filas y columnas) y los tipos de dato de cada columna en los tres formatos y en ambos lenguajes.
3. Crear la columna derivada “ticket” (ventas ÷ clientes) y calcular la venta promedio por sucursal, el ticket promedio, las sucursales con mayor y menor venta y la región con

mayor venta total.

4. Reportar las diferencias encontradas entre formatos y entre lenguajes.

3 · Desarrollo de la práctica

3.1. Instalación de R y Python

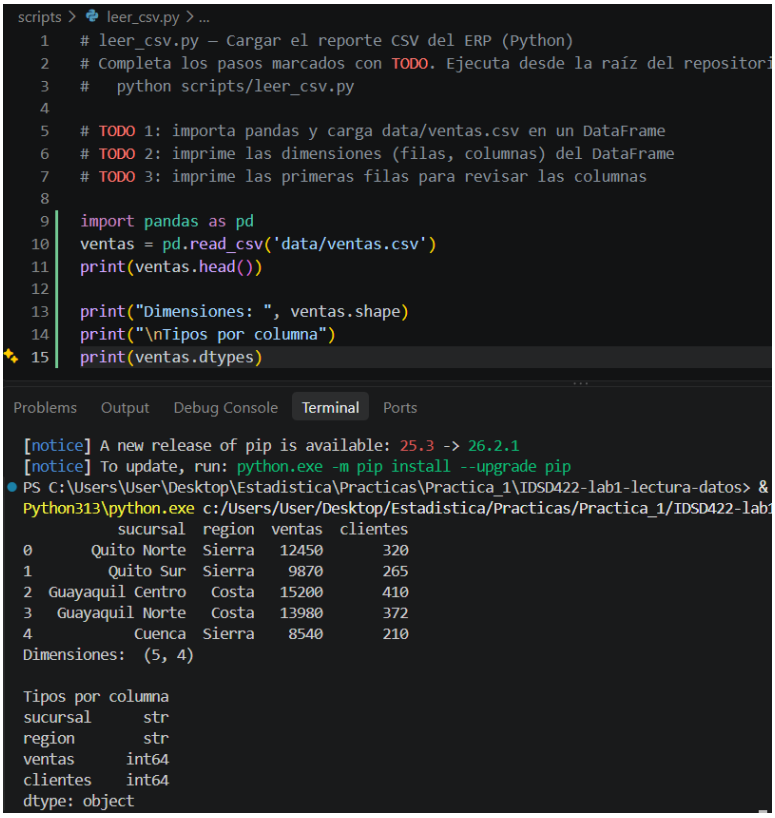
Al iniciar esta práctica fue necesario en mi caso instalar las librerías y lenguajes que vamos a usar, en mi caso el lenguaje Python ya lo poseía previamente instalado, por lo que mis esfuerzos se centraron más en instalar R.

Para instalar R, ingrese al Cran principal de R Project, y descargue su versión para Windows siendo en mi caso la 4.6.1. Mas al instalrlo tuve antes que comenzar a programar instalar la libreria “lenguageserver” esto para autocompletar algunos lexemas.

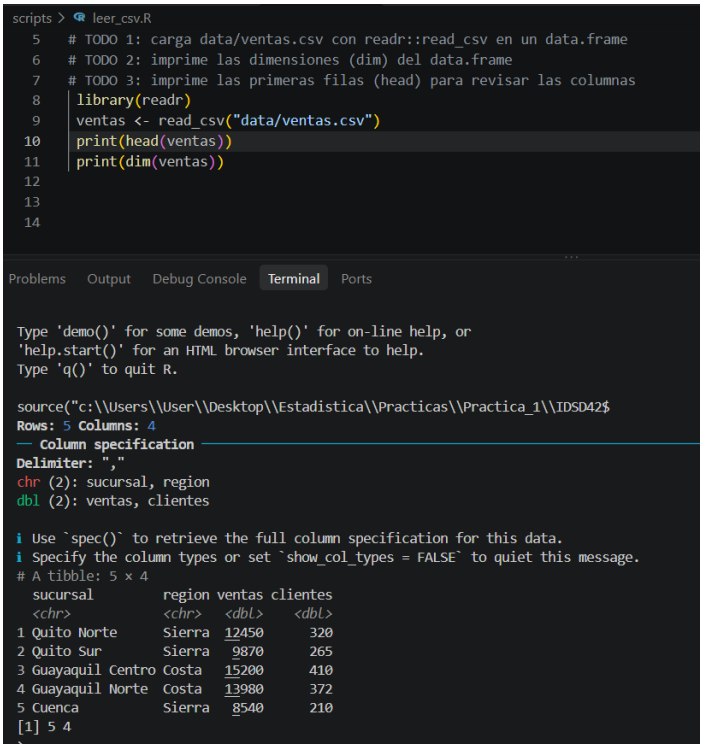
3.2. Programación de la lectura

Lo primero fue comparar que todos los archivos contuviesen la misma información, y para ello se programó en Visual Studio Code las lecturas dedicadas a cada tipo de archivo. Estas, por lo menos con Python, no fueron tan diferentes entre sí más aun así vamos a detallar algunas de las diferencias que conseguí notar.

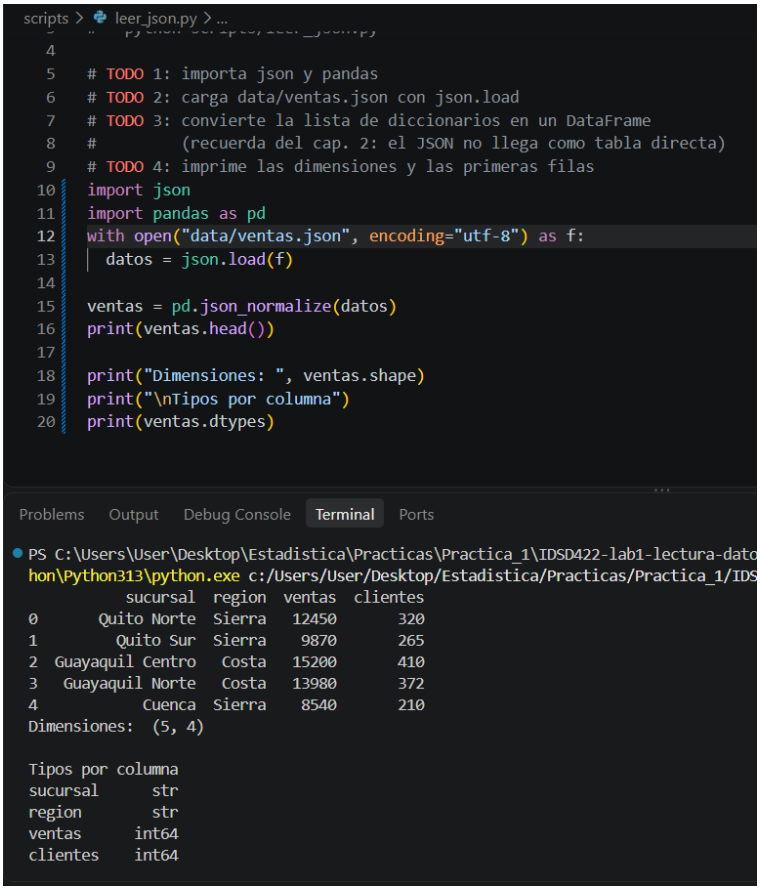
En Python, para leer el archivo .csv se utilizó la librería “pandas”, y a esta lectura la guardamos en una variable, con ello fue fácil el obtener la información que deseábamos, siendo esto sus dimensiones, tipos y primeras filas. Para ello se usó `pd.read_csv()`, y luego `head()`, `shape` y `dtypes`. En la captura se ve que el resultado es un DataFrame de 5 filas y 4 columnas (sucursal, region, ventas y clientes), donde sucursal y region son texto y ventas y clientes son enteros (`int64`).



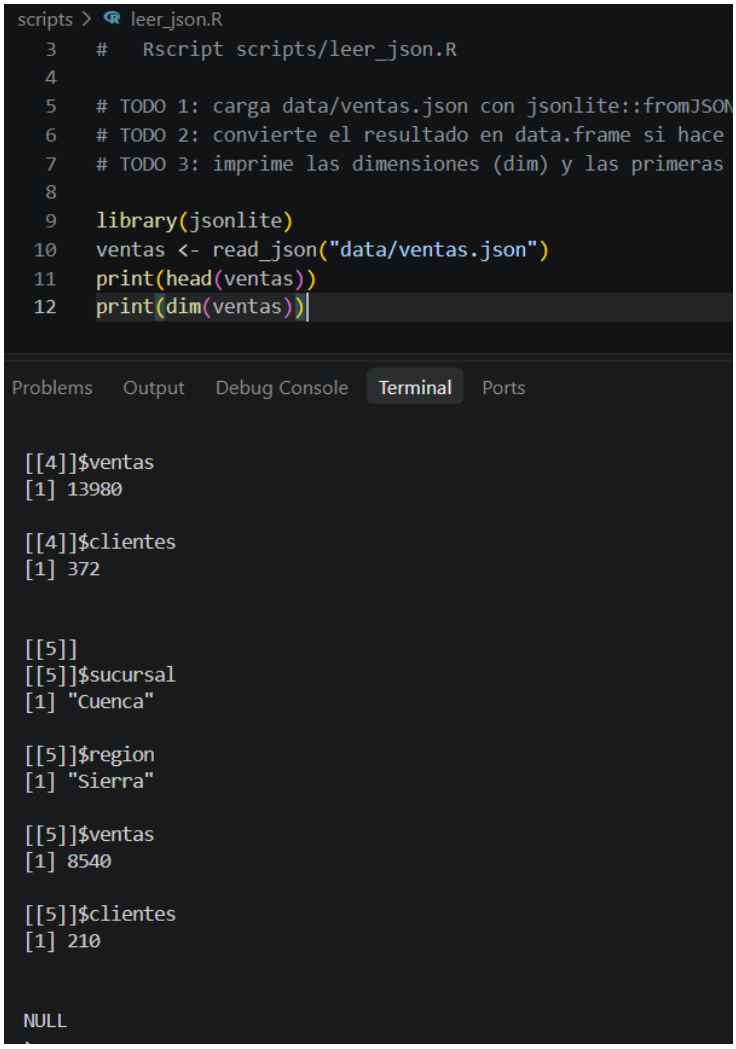
En R, el CSV se leyó con la librería “readr” y su función `read_csv()`, que devuelve un “tibble” (una versión moderna del data.frame). A diferencia de pandas, readr avisa en la consola qué tipos detectó en cada columna: dos de texto (`chr`: sucursal y region) y dos numéricas (`dbl`: ventas y clientes). Las dimensiones salen como `[1] 5 4`, igual que en Python. Una diferencia que noté es que R tomó ventas y clientes como decimales (`dbl`) mientras que pandas las leyó como enteros (`int64`).



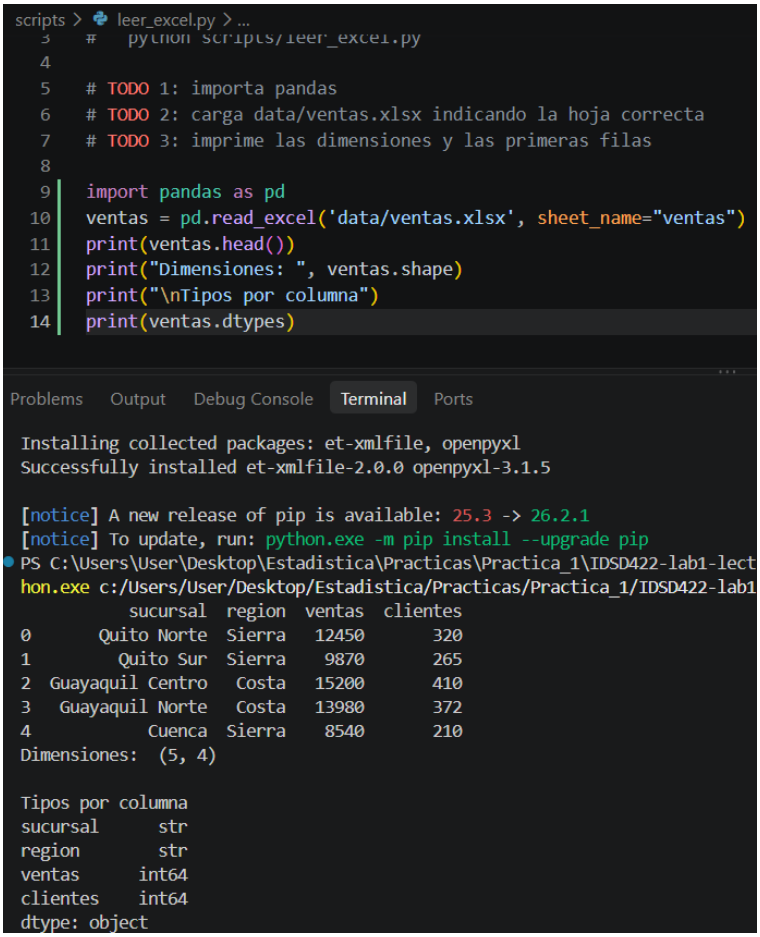
Después de esto, para leer el archivo .json, hay que tener en cuenta que un JSON no llega como tabla directa, como en el caso del CSV, por ello es necesario usar la librería “json”: con `json.load()` se abre el archivo y se obtiene una lista de diccionarios, y luego `pd.json_normalize()` convierte esa lista en un DataFrame. El resultado, que se ve en la captura, fue la misma tabla de 5 filas y 4 columnas, con los mismos tipos que el CSV (texto en sucursal y region, `int64` en ventas y clientes).



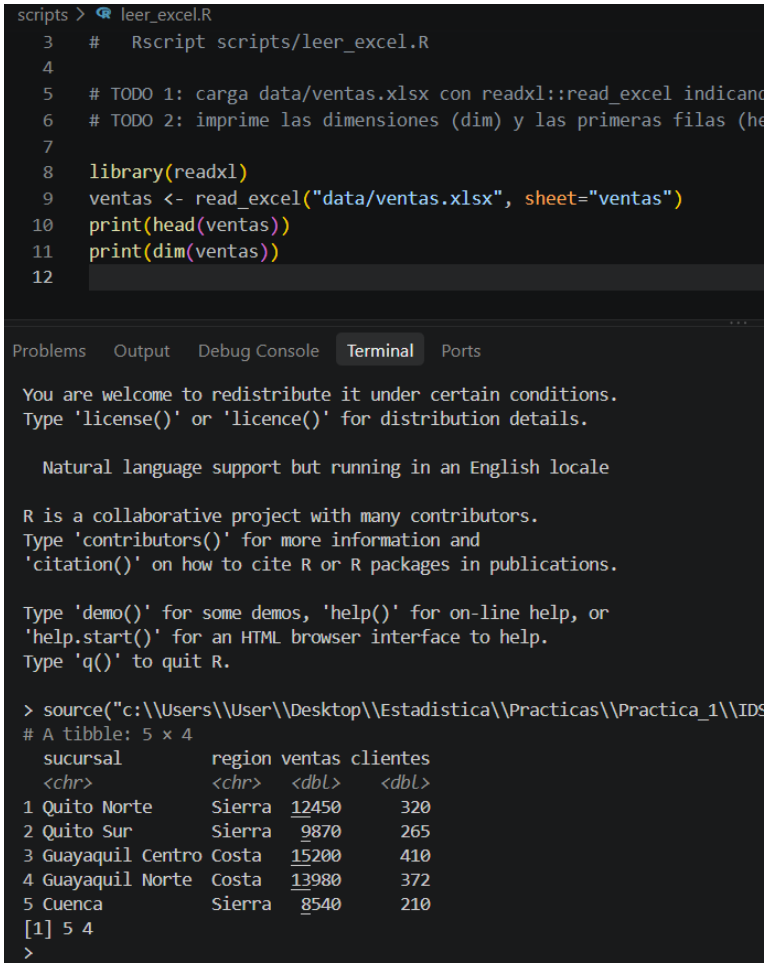
En R, el JSON se leyó con la librería “jsonlite” y la función `read_json()`. Aquí el resultado fue distinto al de Python: como se ve en la captura, R entregó una lista anidada, donde cada sucursal es un elemento con sus respectivos campos. Por eso `dim(ventas)` devolvió `NULL` en lugar de `5 4`. Esto confirma que el JSON no llega como tabla directa: igual que en Python con `json_normalize()`, en R hay que convertir la lista en un data.frame, por ejemplo pidiéndole a la función que simplifique el resultado (`simplifyVector = TRUE`).



Para leer un Excel en Python se usó `pd.read_excel()`, indicando la hoja correcta con `sheet_name="ventas"`. Para que funcione fue necesario instalar la librería “openpyxl”, que pandas utiliza por detrás para abrir archivos .xlsx. El resultado fue, una vez más, la misma tabla de 5 filas y 4 columnas, con los mismos tipos que el CSV y el JSON.



En R, el Excel se leyó con la librería “readxl” y la función `read_excel()`, indicando la hoja con `sheet = "ventas"`. Devolvió un tibble de 5 × 4 con los mismos datos y, igual que con el CSV, las columnas numéricas quedaron como `dbl`.



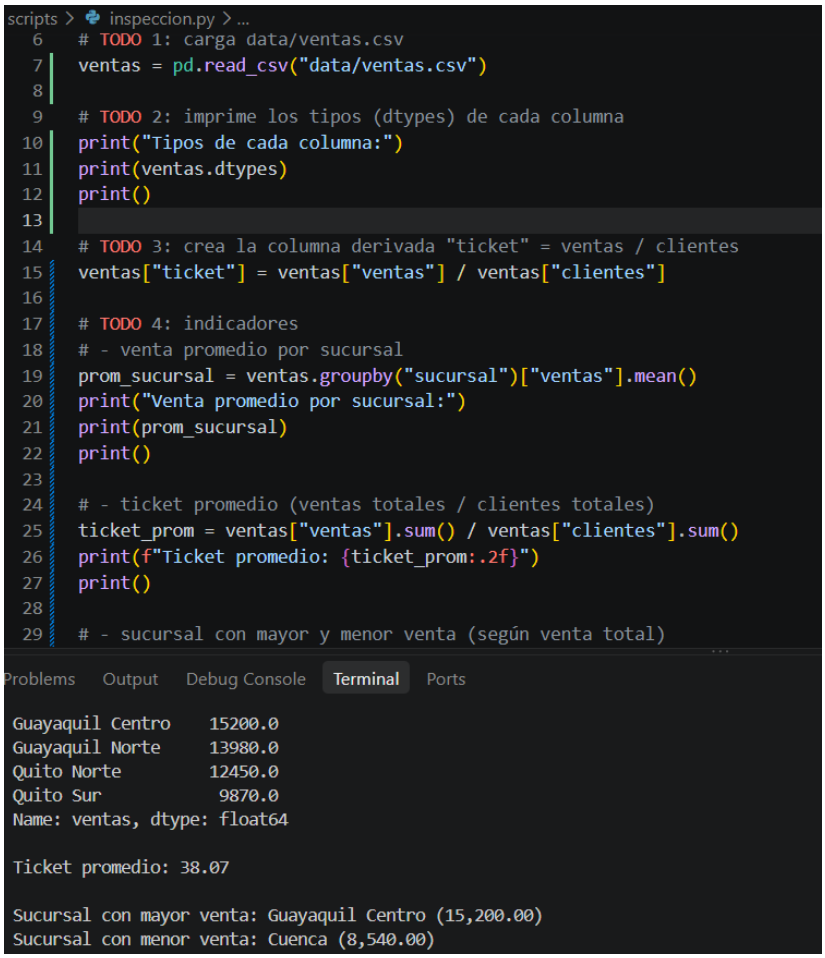
Con esto se comprobó que los tres reportes, leídos tanto en Python como en R, contienen la misma información: 5 filas (una por sucursal) y 4 columnas (sucursal, region, ventas y clientes) con los mismos valores. Las diferencias estuvieron en los tipos que asigna cada lenguaje (enteros en pandas y decimales en R) y en que el JSON en R llega como lista y necesita convertirse a tabla.

3.3. Programación de la inspección

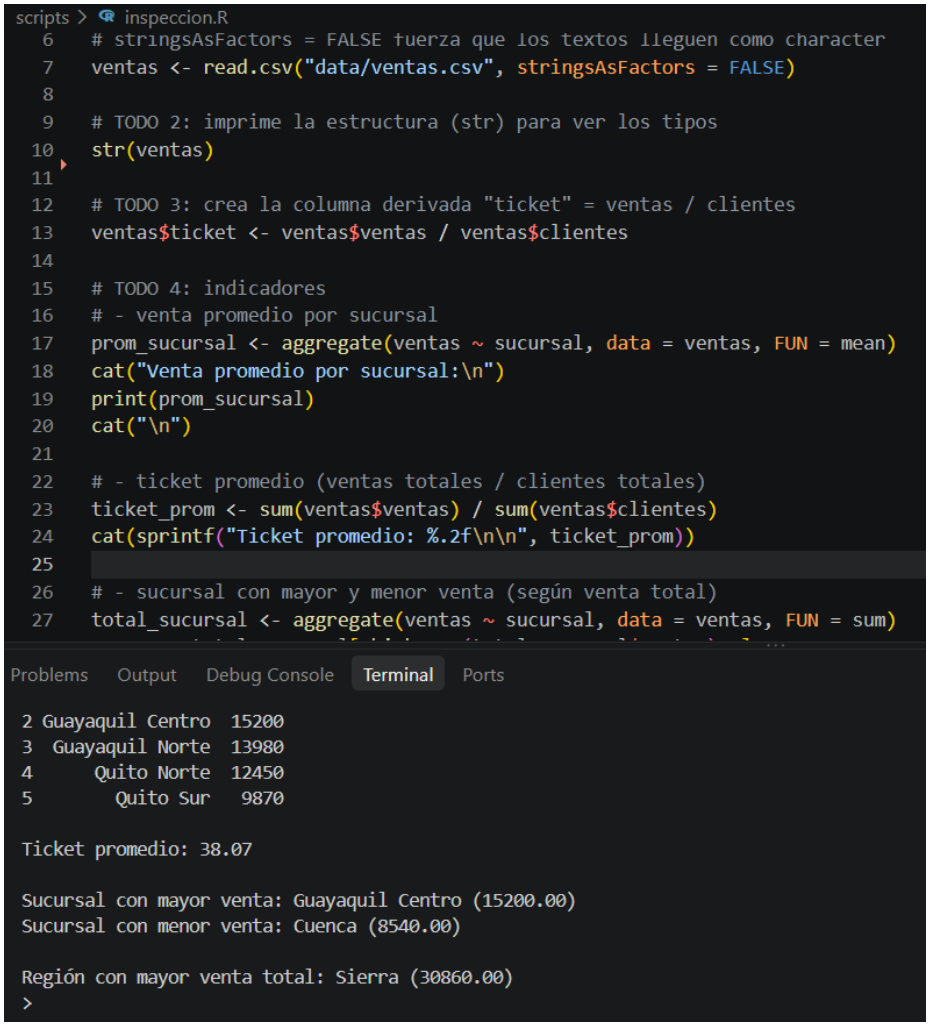
Con los datos ya cargados, el siguiente paso fue inspeccionarlos. Para esto se trabajó con el script de inspección, ubicado en la carpeta `scripts` del repositorio. Ambos siguen los mismos cuatro pasos de la guía: cargar `data/ventas.csv`, revisar el tipo de dato de cada columna, crear la columna derivada "ticket" (ventas ÷ clientes) y calcular los indicadores que pidió la gerencia: la venta promedio por sucursal, el ticket promedio, las sucursales con mayor y menor venta, y la región con mayor venta total.

En Python, los tipos de cada columna se revisaron con `ventas.dtypes`, la columna derivada se creó con `ventas["ticket"] = ventas["ventas"] / ventas["clientes"]` y la venta promedio por sucursal con `groupby("sucursal")["ventas"].mean()`. Los demás indicadores se calcularon

con sumas directas sobre las columnas. En la siguiente captura se ve el código y su salida en la terminal.



En R el análisis es el mismo, pero cambia la forma de escribirlo. Al leer el CSV se usó `stringsAsFactors = FALSE` para que los textos lleguen como “character” y no como factores; la estructura y los tipos se imprimieron con `str(ventas)`, que además muestra una vista previa de los valores; la columna derivada se creó con el operador `$` y los agrupamientos se hicieron con `aggregate()`, que recibe una fórmula como `ventas ~ sucursal`.



Ambos lenguajes entregaron exactamente las mismas cifras. El ticket promedio fue de 38.07, la sucursal con mayor venta fue Guayaquil Centro (15,200.00) y la de menor venta fue Cuenca (8,540.00). En R, además, se obtuvo que la región con mayor venta total fue la Sierra, con 30,860.00.

4 · ¿Qué aprendí con esta práctica?

Con esta práctica aprendí a leer un mismo reporte de ventas desde tres formatos distintos (CSV, JSON y Excel) tanto en Python como en R, algo que antes no había hecho de forma comparada. También aprendí a revisar rápidamente la estructura de una tabla (filas, columnas y tipos de dato), a crear una columna derivada como el ticket y a calcular indicadores agrupando por sucursal y por región, con groupby() en pandas y con aggregate() en R.

Lo más difícil fue la configuración inicial y el JSON en R. Para instalar R tuve que añadir la librería "languageserver" para tener autocompletado en Visual Studio Code, y para leer el Excel en Python tuve que instalar "openpyxl". Con el JSON, en Python json_normalize() lo convirtió en tabla sin problema, pero en R read_json() me devolvió una lista anidada y dim() dio NULL. Eso me hizo entender que un JSON no es una tabla por defecto y que hay que convertirlo; la documentación de

jsonlite (Ooms, s. f.) indica que el resultado se puede simplificar con simplifyVector = TRUE.
La próxima vez probaría la conversión del JSON en R desde el inicio, numeraría y describiría las capturas mientras las tomo, y confirmaría con dim() que los tres archivos coinciden antes de calcular indicadores.

5 · Conclusiones

6 · Bibliografía

Escuela Politécnica Nacional. (2026). *IDSD422-lab1-lectura-datos* [Repositorio de GitHub]. https://github.com/Jeriel2/IDSD422-lab1-lectura-datos

Ooms, J. (s. f.). *jsonlite: A simple and robust JSON parser and generator for R*. CRAN. https://cran.r-project.org/package=jsonlite

pandas development team. (s. f.). *pandas documentation*. https://pandas.pydata.org/docs/

R Core Team. (s. f.). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. https://www.R-project.org/

Wickham, H., & Bryan, J. (s. f.). *readxl: Read Excel files*. https://readxl.tidyverse.org/

Wickham, H., Hester, J., & Bryan, J. (s. f.). *readr: Read rectangular text data*. https://readr.tidyverse.org/

Declaración de uso de IA. Usé Claude (Anthropic) como apoyo para redactar las descripciones de las capturas, los objetivos, la interpretación de resultados y las conclusiones. El código lo ejecuté yo, las capturas son de mi propia ejecución y las cifras del informe corresponden a la salida de mis scripts.